



Chapter 4

Lists

Data Structures and Algorithms

Dr. Nguyen Ho Man Rang
Faculty of Computer Science and Engineering
University of Technology, VNU-HCM

- **L.O.2.1** - Depict the following concepts: (a) array list and linked list, including single link and double links, and multiple links.
- **L.O.2.2** - Describe storage structures by using pseudocode for: (a) array list and linked list, including single link and double links, and multiple links.
- **L.O.2.3** - List necessary methods supplied for list and describe them using pseudocode.
- **L.O.2.4** - Implement list using C/C++.



- **L.O.2.5** - Use list for problems in real-life, and choose an appropriate implementation type (array vs. link).
- **L.O.2.6** - Analyze the complexity and develop experiment (program) to evaluate the efficiency of methods supplied for list.
- **L.O.8.4** - Develop recursive implementations for methods supplied for the following structures: list.
- **L.O.1.2** - Analyze algorithms and use Big-O notation to characterize the computational complexity of algorithms composed by using the following control structures: sequence, branching, and iteration (not recursion).



Contents

- ① Linear list concepts
- ② Array implementation
- ③ Singly linked list
- ④ Other linked lists
- ⑤ Comparison of implementations of list



Linear list concepts

Array
implementation

Singly linked list

Other linked lists

Comparison of
implementations of
list



Linear list concepts

Array
implementation

Singly linked list

Other linked lists

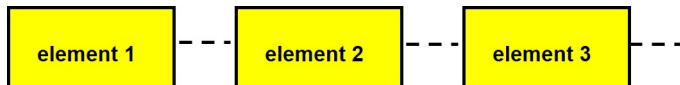
Comparison of
implementations of
list

Linear list concepts



Definition

A linear list is a finite, ordered sequence of data items known as elements. "Ordered" in this definition means that each element has a position in the list.



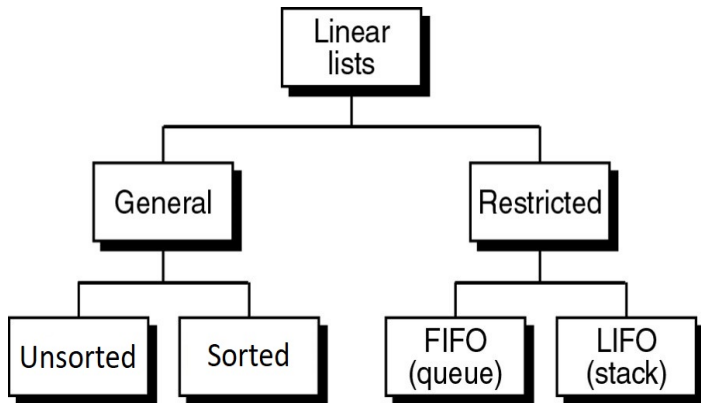
Linear list concepts

Array
implementation

Singly linked list

Other linked lists

Comparison of
implementations of
list





General list:

- No restrictions on which operation can be used on the list.
- No restrictions on where data can be inserted/deleted.
- **Unsorted list**: data are not arranged in particular order.
- **Sorted list**: data are arranged according to a key.

Linear list concepts

Array
implementation

Singly linked list

Other linked lists

Comparison of
implementations of
list



Restricted list:

- Only some operations can be used on the list.
- Data can be inserted/deleted only at the ends of the list.
- **Queue**: FIFO (First-In-First-Out).
- **Stack**: LIFO (Last-In-First-Out).

Linear list concepts

Array
implementation

Singly linked list

Other linked lists

Comparison of
implementations of
list



Definition

A list of elements of type T is a finite, ordered sequence of elements of T .

Basic concepts:

- A list is **empty** when it contains no elements.
- The number of elements currently stored is called the **length (size)** of the list.
- The beginning of the list is called the **head**, the end of the list is called the **tail**.

Linear list concepts

Array
implementation

Singly linked list

Other linked lists

Comparison of
implementations of
list



Basic operations:

- Construct a list, leaving it empty.
- Insert an element.
- Remove an element.
- Search an element.
- Retrieve an element.
- Traverse the list, performing a given operation on each element.

Linear list concepts

Array
implementation

Singly linked list

Other linked lists

Comparison of
implementations of
list



Extended operations:

- Determine whether the list is **empty** or not.
- Determine whether the list is **full** or not.
- Find the **size** of the list.
- **Clear** the list to make it empty.
- **Replace** an element with another element.
- **Merge** two ordered list.
- **Append** an unordered list to another.

Linear list concepts

Array
implementation

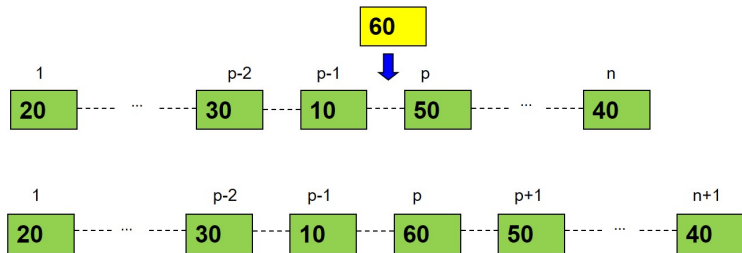
Singly linked list

Other linked lists

Comparison of
implementations of
list



- Insert an element at a specified position p in the list
 - Only with *General Unsorted List*.



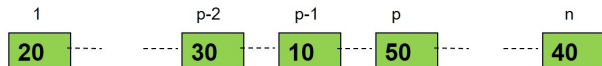
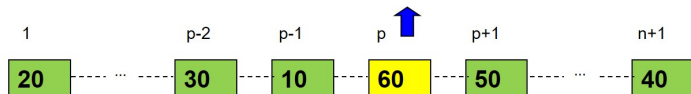
Any element formerly at position p and all later have their position numbers increased by 1.

- **Insert an element with a given data**
 - With *General Unsorted List*: can be made at any position in the list (at the beginning, in the middle, at the end).
 - With *General Sorted List*: data must be inserted so that the ordering of the list is maintained (searching appropriate position is needed).
 - With *Restricted List*: depend on it own definition (FIFO or LIFO).



Removal

- Remove an element at a specified position p in the list
 - With *General Unsorted List* and *General Sorted List*.



The element at position p is removed from the list, and all subsequent elements have their position numbers decreased by 1.

- Retrieve an element at a specified position p in the list
 - With *General Unsorted List* and *General Sorted List*.



retrieved data = 60

All elements remain unchanged.



- **Remove/ Retrieve an element with a given data**
 - With *General Unsorted List* and *General Sorted List*: Searching is needed in order to locate the data being deleted/ retrieved.



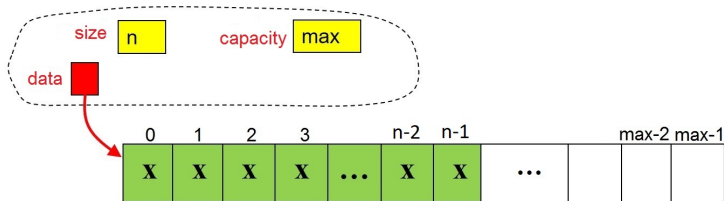
- **Insertion** is successful when the list is not full.
- **Removal, Retrieval** are successful when the list is not empty.





Array implementation

Dynamically Allocated Array



```
List // Contiguous Implementation of List
// number of used elements (mandatory)
size <integer>

// (Dynamically Allocated Array)
data <dynamic array of <DataType> >

capacity <integer>
End List
```



Dynamic Array: Implementation in C++

```
class DynamicArray {  
private:  
    int size;  
    int capacity;  
    int *storage;  
  
public:  
    DynamicArray() {  
        capacity = 10;  
        size = 0;  
        storage = new int[capacity];  
    }  
}
```



Dynamic Array: Implementation in C++

```
DynamicArray(int capacity) {  
    this->capacity = capacity;  
    size = 0;  
    storage = new int[capacity];  
}  
  
~DynamicArray() {  
    delete[] storage;  
}
```



Dynamic Array: Implementation in C++

```
void setCapacity(int );
void ensureCapacity(int );
void pack();
void trim();

void rangeCheck(int );
void set(int , int );
int get(int );
void removeAt(int );
void insertAt(int , int );

void print();
};
```



Dynamic Array: Implementation in C++

```
void DynamicArray::setCapacity(  
int newCapacity) {  
    int *newStorage = new int[newCapacity];  
    memcpy(newStorage, storage, sizeof(int) * size);  
    capacity = newCapacity;  
    delete[] storage;  
    storage = newStorage;  
}
```



Dynamic Array: Implementation in C++

```
void DynamicArray::ensureCapacity(  
    int minCapacity) {  
    if (minCapacity > capacity) {  
        int newCapacity = (capacity*3)/2 + 1;  
        if (newCapacity < minCapacity)  
            newCapacity = minCapacity;  
        setCapacity(newCapacity);  
    }  
}
```



Dynamic Array: Implementation in C++

```
void DynamicArray::pack() {  
    if (size <= capacity / 2) {  
        int newCapacity = (size * 3) / 2 + 1;  
        setCapacity(newCapacity);  
    }  
}  
  
void DynamicArray::trim() {  
    int newCapacity = size;  
    setCapacity(newCapacity);  
}
```



Dynamic Array: Implementation in C++

```
void DynamicArray::rangeCheck(int index) {  
    if (index < 0 || index >= size)  
        throw "Index out of bounds!";  
}  
  
void DynamicArray::set(int index,  
                       int value) {  
    rangeCheck(index);  
    storage[index] = value;  
}  
  
int DynamicArray::get(int index) {  
    rangeCheck(index);  
    return storage[index];  
}
```



Dynamic Array: Implementation in C++

```
void DynamicArray::insertAt(int index ,
                           int value) {
    if (index < 0 || index > size)
        throw "Index out of bounds!";

    ensureCapacity(size + 1);

    int moveCount = size - index;
    if (moveCount != 0)
        memmove(storage + index + 1,
                storage + index ,
                sizeof(int) * moveCount);
    storage[index] = value;
    size++;
}
```



Dynamic Array: Implementation in C++

```
void DynamicArray::removeAt(int index) {  
    rangeCheck(index);  
    int moveCount = size - index - 1;  
    if (moveCount > 0)  
        memmove(storage + index ,  
                storage + (index + 1),  
                sizeof(int) * moveCount);  
    size --;  
    pack();  
}
```



Dynamic Array: Using

```
void DynamicArray::print() {
    for (int i=0; i<this->size; i++) {
        cout << storage[i] << " ";
    }
}

int main() {
    cout << "Dynamic Array" << endl;
    DynamicArray* da = new DynamicArray(10);
    da->insertAt(0, 55);
    // ...
    da->print();
    return 0;
}
```



Contiguous Implementation of List

In processing a contiguous list with n elements:

- **Insert** and **Remove** operate in time approximately proportional to n (**require physical shifting**).
- **Clear**, **Empty**, **Full**, **Size**, **Replace**, and **Retrieve** in constant time.

